Making Deep Networks Trainable

Initialization, activations, normalization and residual connections

Prof. Nipun Batra

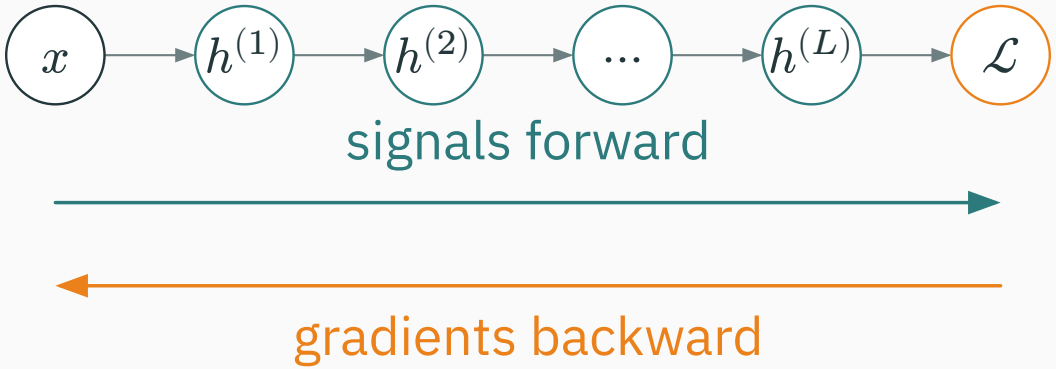
ES 667 · Deep Learning · IIT Gandhinagar

Why depth breaks naive networks

The spine of this lecture

V

A deep network must preserve useful **signals** forward and useful **gradients** backward.



everything today keeps these two flows from collapsing or exploding

We have the ingredients — so why stall?

We can already build a net (neurons, nonlinearities), get exact **gradients** (backprop), and **use** them (SGD / Adam). So why not stack 100 layers?

Stack many layers with textbook choices (sigmoid, small random weights) and training **stalls or blows up**. A 100-layer plain net often does **worse** than a 10-layer one — not overfitting, it will not **optimize**.

the problem is in the **forward and backward signals**, before optimization even begins

The deep-net experiment

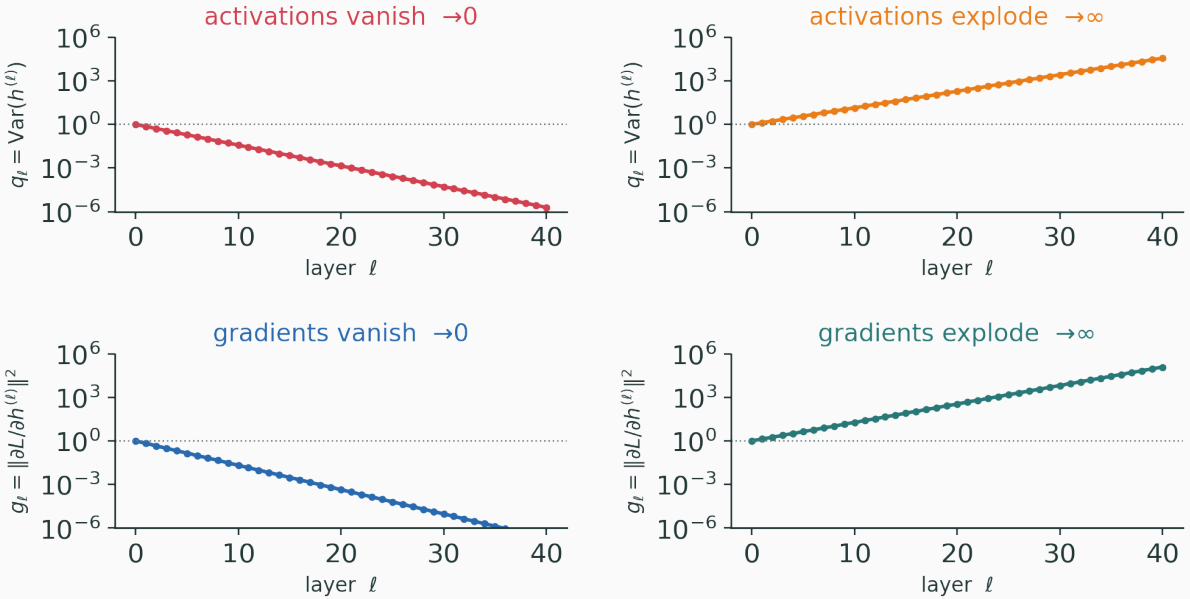
V

Push a signal through L layers and **track two numbers** per layer:

$$q_\ell = \mathrm{Var}\big(h^{(\ell)}\big) \quad \text{(activation scale, forward)}, \qquad g_\ell = \big\|\partial \mathcal{L}/\partial h^{(\ell)}\big\|^2 \quad \text{(gradient scale, backward)}$$

With naive init, **both** q_ℓ and g_ℓ drift **exponentially** with depth — either $\to 0$ (vanish) or $\to \infty$ (explode). A healthy net keeps them $\approx$ constant.

Four failure modes



activations and gradients can each **vanish** or **explode** — four ways depth breaks

The forward and backward recurrences

D

Forward — each layer applies a linear map then a nonlinearity:

$$z^{(\ell)} = W_\ell \, h^{(\ell-1)}, \qquad h^{(\ell)} = \varphi\big(z^{(\ell)}\big).$$

Backward — backprop pushes the adjoint through the transpose and the local slope:

$$\overline{h}^{(\ell-1)} = W_\ell^\top \Big(\overline{h}^{(\ell)} \odot \varphi'\big(z^{(\ell)}\big)\Big).$$

$\overline{h}^{(\ell)} := \partial \mathcal{L} / \partial h^{(\ell)}$ — the same W and φ' control **both** directions

A product of Jacobians

D

Compose the per-layer Jacobians $J_\ell = \partial h^{(\ell)} / \partial h^{(\ell-1)}$:

$$\frac{\partial h^{(L)}}{\partial x} = J_L \, J_{L-1} \cdots J_1 .$$

A product of L matrices is governed by its **singular values** s:

$s < 1$ at each layer $\Rightarrow$ product $\to 0 \Rightarrow$ **vanishing**.

$s > 1$ at each layer $\Rightarrow$ product $\to \infty \Rightarrow$ **exploding**.

we need each layer's Jacobian to have singular values ≈ 1

Four interventions, one goal

V

Problem	Intervention
wrong **starting** scale of signals	**initialization** — set $\mathrm{Var}(w)$
local slope kills the gradient	**activations** — shape the Jacobian φ'
scale **drifts during** training	**normalization** — re-standardize each layer
gradient path too **long**	**residual connections** — add identity shortcuts

all four exist to keep q_ℓ and g_ℓ near constant with depth

Activations and gradient flow

What an activation must do

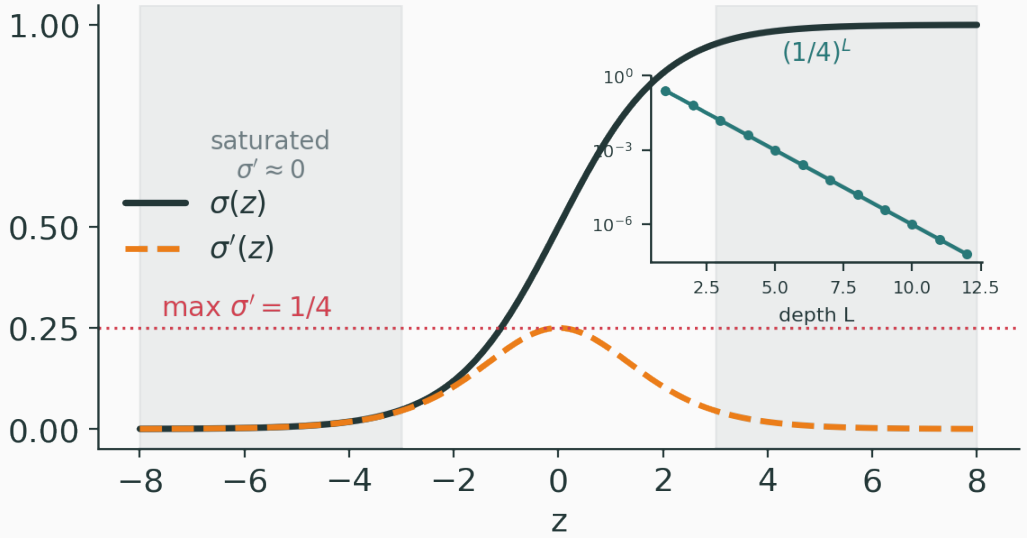
A good activation φ has to be three things at once:

- **nonlinear** — else a stack of layers collapses to one linear map;
- **gradient-friendly** — φ' not tiny, so backprop survives depth;
- **cheap** — evaluated billions of times per step.

If φ' is small over most of its range, deep gradients **vanish** — the activation itself becomes the bottleneck.

Sigmoid

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1-\sigma(z)).$$



the derivative peaks at $1/4$ and is ≈ 0 once $|z|$ is large — **saturation**

Why sigmoid breaks depth

D

The peak slope is only $1/4$, so a chain of sigmoids can only **shrink** gradients:

$$\prod_{\ell=1}^{L} \sigma'(z_\ell) \leq \left(\frac{1}{4}\right)^{L}.$$

For $L = 10$: $(1/4)^{10} \approx 9.5 \times 10^{-7}$ — gradients are essentially gone.

Sigmoid is also **not zero-centred** (outputs in $(0, 1)$): activations are all-positive, biasing gradients and slowing learning.

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \qquad \tanh'(z) = 1 - \tanh^2(z).$$

- **zero-centred** — outputs in $(-1, 1)$, fixing sigmoid's bias problem;
- max derivative is 1 (not $1/4$) — gradients survive a bit longer.

Still **saturates** for large $|z|$: $\tanh' \to 0$. Better than sigmoid, but deep tanh stacks still vanish.

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{ReLU}'(z) = \mathbb{1}[z > 0] \in \{0, 1\}.$$

- **no positive saturation** — for $z > 0$ the slope is exactly 1, so gradients pass **unattenuated**;
- **sparse** — about half the units output 0;
- **cheap** — a single comparison.

the default hidden activation for MLPs and CNNs

If a unit's pre-activation is **always** negative, it is stuck:

$$z < 0 \quad \Rightarrow \quad h = 0, \quad \partial h / \partial z = 0 \quad (\text{no gradient, ever}).$$

Causes: too-large learning rate, bad init, or a large **negative bias** pushing z below 0.

A **dead** ReLU never recovers — its gradient is zero, so it is never updated. Watch the **fraction of zero activations** as a diagnostic.

Leaky ReLU and PReLU

Give the negative side a small slope so the unit can recover:

$$\mathrm{LeakyReLU}(z) = \begin{cases} z & z > 0 \\ azz & z \leq 0, \end{cases} \quad a \approx 0.01.$$

- the negative branch has slope $a > 0 \Longrightarrow$ **nonzero gradient** $\Longrightarrow$ no permanent death;
- **PReLU** makes a a **learned** parameter per channel.

A cheap insurance policy against dead units, at the cost of one extra hyperparameter (or parameter).

Smooth modern activations

D

Gate the input by a smooth probability instead of a hard step:

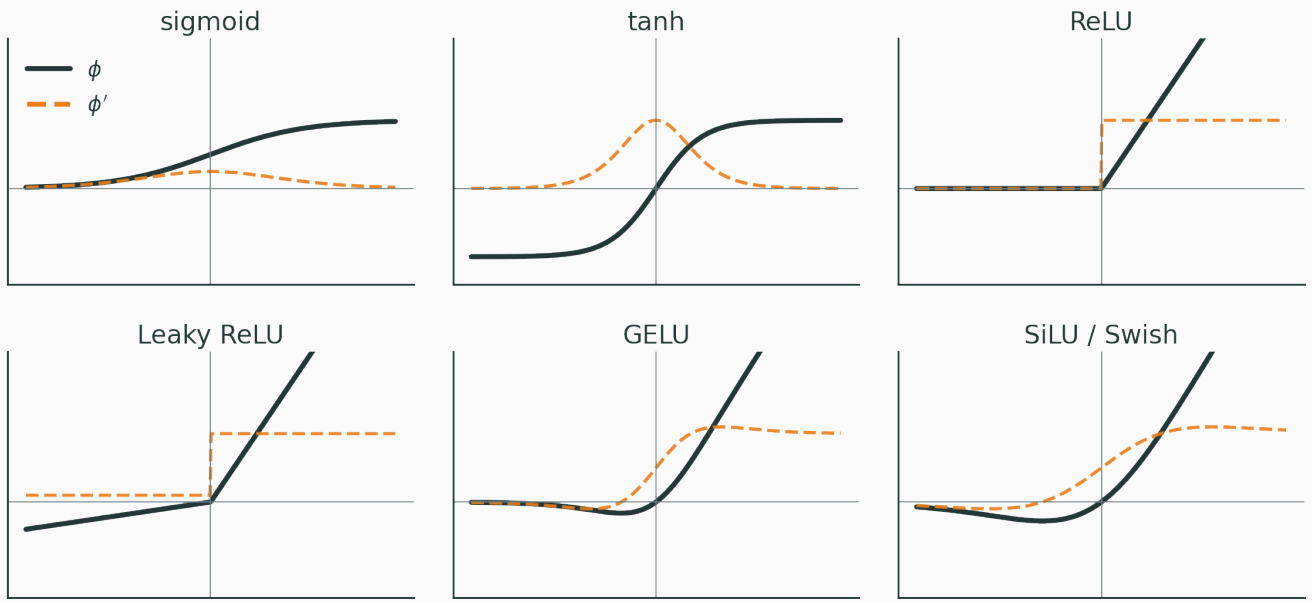
$$\mathrm{GELU}(z) = z\,\Phi(z), \qquad \mathrm{SiLU}(z) = z\,\sigma(z),$$

where Φ is the standard-normal CDF.

- smooth everywhere $\Longrightarrow$ well-behaved second-order behaviour;
- **nonzero gradient** for small negative z (unlike ReLU);
- the default in **Transformers** (GELU) and many modern CNNs (SiLU / Swish).

The activation zoo

V



solid φ, dashed φ' — ReLU/Leaky/GELU/SiLU keep a healthy slope where sigmoid/tanh flatten

Choosing an activation

Activation	0-centred	Saturates	Neg. grad	Typical use
sigmoid	no	both sides	≈ 0	gates, binary output
tanh	yes	both sides	small	older RNN hidden
ReLU	no	no (pos.)	0 (dies)	CNN / MLP default
Leaky / PReLU	no	no	az (alive)	avoid dead units
GELU / SiLU	$\approx$	no	smooth	Transformers, modern CNNs

rule of thumb: ReLU/GELU by default; tanh/sigmoid only where you specifically want a bounded output

Interactive: activation lab

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Swap the activation and sweep depth; watch the gradient that reaches layer 0 collapse for sigmoid/tanh and survive for ReLU/GELU.

Q. A 20-layer sigmoid net barely learns. Name **two** changes that would most help its gradient flow.

Initialization: the starting scale

Why not initialize to all zeros? D

Set every weight equal (e.g. all zero). Two units in a layer see the **same** input and the same weights, so:

$$\text{same activation} \quad \Rightarrow \quad \text{same gradient} \quad \Rightarrow \quad \text{same update} \quad (\text{forever}).$$

Symmetry is never broken — every unit in a layer stays identical, so the layer has the expressive power of **one** unit.

Random weights break the symmetry. **Biases** can safely start at 0 — the random weights already differentiate the units.

Study one pre-activation with n zero-mean, independent inputs and weights:

$$z = \sum_{i=1}^{n} w_i x_i.$$

With $\mathbb{E}[w_i] = 0$ and $w \perp x$:

$$\mathbb{E}[z] = \sum_{i=1}^{n} \mathbb{E}[w_i]\mathbb{E}[x_i] = 0.$$

zero-mean weights $\Rightarrow$ zero-mean pre-activations — now control the variance

Forward-variance derivation

D

Variance of a sum of independent, zero-mean terms:

$$\mathrm{Var}(z) = \sum_{i=1}^{n} \mathrm{Var}(w_i x_i) = \sum_{i=1}^{n} \mathrm{Var}(w)\ \mathrm{Var}(x).$$

All n terms are identical, so

$$\mathrm{Var}(z) = n\,\mathrm{Var}(w)\,\mathrm{Var}(x).$$

$$\boxed{\mathrm{Var}(z) = n_{\text{in}}\,\mathrm{Var}(w)\,\mathrm{Var}(x)}$$

Variance preservation

D

To keep the signal scale **unchanged** across a layer we need $\mathrm{Var}(z) = \mathrm{Var}(x)$:

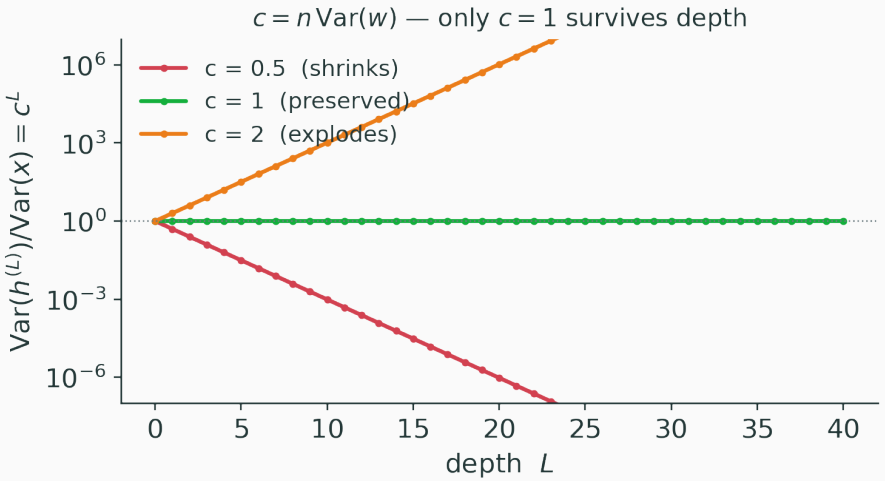
$$n_{\text{in}}\,\mathrm{Var}(w) \approx 1 \quad \Rightarrow \quad \mathrm{Var}(w) \approx \frac{1}{n_{\text{in}}}.$$

Scale the weights by the **fan-in**. Too big amplifies each layer; too small attenuates each layer.

The wrong scale compounds

Write $c = n_{\text{in}}\ \mathrm{Var}(w)$. Across L layers the variance is **multiplied** L times:

$$\mathrm{Var}\big(h^{(L)}\big) \approx c^{L}\,\mathrm{Var}(x).$$



$$c = 0.5, L = 20: \ 0.5^{20} \approx 9.5 \times 10^{-7}$$
$$c = 2, L = 20: \ 2^{20} \approx 1.0 \times 10^{6}$$

only $c = 1$ survives depth; a small miss is fatal after 20 layers

The backward constraint

D

Backprop multiplies by $W^\top$, so the **gradient** variance follows the same law with fan-**out**:

$$\mathrm{Var}\left(\overline{h}^{(\ell-1)}\right) \approx n_{\text{out}}\,\mathrm{Var}(w)\,\mathrm{Var}\left(\overline{h}^{(\ell)}\right).$$

Preserving the **backward** signal wants

$$\mathrm{Var}(w) \approx \frac{1}{n_{\text{out}}}.$$

Forward wants $1/n_{\text{in}}$, backward wants $1/n_{\text{out}}$ — and usually $n_{\text{in}} \neq n_{\text{out}}$. We cannot satisfy both exactly.

Compromise between the two constraints — take the **harmonic-style** average:

$$\mathrm{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

Two equivalent samplers:

$$w \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right), \qquad w \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right).$$

Xavier balances forward and backward variance — designed for tanh / linear layers

Why ReLU changes the constant

D

Xavier assumes φ is linear near 0. ReLU **zeros out half** the inputs:

$$h = \mathrm{ReLU}(z) \quad \Rightarrow \quad \mathbb{E}[h^2] \approx \frac{1}{2}\mathbb{E}[z^2].$$

So a ReLU layer **halves** the signal variance — Xavier's c becomes ≈ 0.5, and the signal decays.

We must put the lost factor of 2 **back** into the weight variance.

He / Kaiming initialization

D

Demand variance preservation **through** the ReLU:

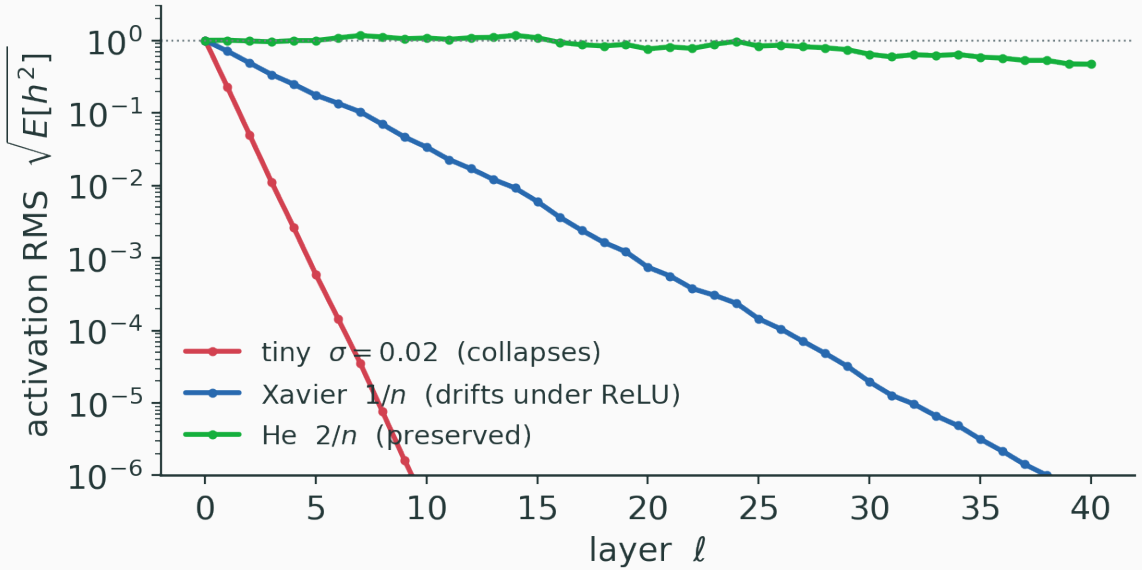
$$\frac{1}{2}\, n_{\text{in}}\, \text{Var}(w) \approx 1 \quad \Rightarrow \quad \text{Var}(w) = \frac{2}{n_{\text{in}}}, \qquad \text{Std}(w) = \sqrt{\frac{2}{n_{\text{in}}}}.$$

He init: $\text{Std}(w) = \sqrt{2/n_{\text{in}}}$ **— the default for ReLU networks**

Generalized for a leaky slope a:

$$\text{Var}(w) = \frac{2}{(1 + a^2)\, n_{\text{in}}}.$$

Signal flow under three inits



He holds the activation scale flat through a deep ReLU net; a tiny σ collapses; Xavier drifts down

Fully worked init example

D

Layer with $n_{\text{in}} = 100$, inputs $x \sim \mathcal{N}(0, 1)$ so $\text{Var}(z) = 100 \cdot \text{Var}(w)$:

Scheme	$\text{Var}(w)$	$\text{Var}(z)$	Effect
(A) $\text{Std} = 0.01$	10^{-4}	0.01	shrinks $100\times$ — vanishes
(B) $\text{Std} = 1$	1	100	explodes $100\times$
(C) Xavier $1/100$	0.01	1	preserved (linear)
(D) He $2/100$	0.02	2	pre-ReLU 2, post-ReLU $\mathbb{E}[h^2] \approx 1$

(A)/(B) are off by $100\times$ in one layer; (C)/(D) preserve the scale — all verified numerically

Initialization for convolutions

D

For a conv layer the fan-in counts **every input that feeds one output**:

$$n_{\text{in}} = k_h \cdot k_w \cdot C_{\text{in}}.$$

A 3×3 kernel over 64 input channels:

$$n_{\text{in}} = 3 \cdot 3 \cdot 64 = 576, \qquad \text{Std}_{\text{He}} = \sqrt{\frac{2}{576}} \approx 0.0589.$$

same rule, correct fan-in — the constant just changes

Initialization is not a guarantee

I

Good init makes training **start** in a healthy regime — but only at $t = 0$.

The derivations assume zero-mean, independent, unit-scale inputs. During training weights **shift**, correlations build, and the guarantees **erode**.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Sweep the init scale and watch the per-layer activation std light up green (healthy) or red (vanishing / exploding) across a deep stack.

Normalization: scale during training

Init only controls the first step

D

Initialization sets the signal scale at $t = 0$; then optimization moves the weights and the guarantee erodes. We need to **re-standardize** activations at **every** step. For a batch of a quantity x, subtract the mean and divide by the std:

$$\mu = \frac{1}{B}\sum_{i=1}^{B} x_i, \quad \sigma^2 = \frac{1}{B}\sum_{i=1}^{B}(x_i - \mu)^2, \quad \hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}.$$

$\hat{x}$ has mean 0 and variance 1 — a controlled scale, by construction

$\varepsilon \approx 10^{-5}$ guards against dividing by zero

Why the learnable γ, β

D

Forcing mean 0, variance 1 can be **too** restrictive. Add a learnable affine map back:

$$y = \gamma\,\hat{x} + \beta.$$

- γ rescales, β re-shifts — both **learned**;
- the network can **recover the identity** ($\gamma = \sqrt{\sigma^2 + \varepsilon}$, $\beta = \mu$) if normalizing hurt.

normalize for a stable scale, then let the model choose the scale it wants

Which axis do we normalize over?

Given a batch of activations $X \in \mathbb{R}^{B \times D}$ (B examples, D features), we can normalize:

- **down each feature** (over the batch) → **BatchNorm**;
- **across each example** (over the features) → **LayerNorm**.

The **axis you reduce over** is the entire difference between the normalization schemes.

Normalize **each feature** j using statistics **over the batch**:

$$\mu_j = \frac{1}{B}\sum_{i=1}^{B} x_{ij}, \quad \sigma_j^2 = \frac{1}{B}\sum_{i=1}^{B}(x_{ij} - \mu_j)^2, \quad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

One $(\mu_j, \sigma_j, \gamma_j, \beta_j)$ **per feature**. Each column of the batch matrix is standardized independently.

BatchNorm normalizes down each feature

orange column = one feature j, normalized using its mean/variance **over the whole batch**.

rows = examples in the batch (B)

columns = features (D)

reduce **down** each column

BatchNorm for a conv layer

D

Treat every spatial location as another sample. Normalize **per channel** over (B, H, W):

$$\mu_c, \sigma_c^2 \quad \text{computed over all } B \cdot H \cdot W \text{ activations of channel } c.$$

One statistic **per channel**, shared across space — this is why BatchNorm is so natural in CNNs.

Train vs inference

D

At **training** time BatchNorm uses the **batch** statistics. At **inference** a single example has no batch — so keep a **running average**:

$$\mu_{\text{run}} \leftarrow \rho\,\mu_{\text{run}} + (1-\rho)\,\mu_B, \qquad \sigma^2_{\text{run}} \leftarrow \rho\,\sigma^2_{\text{run}} + (1-\rho)\,\sigma^2_B.$$

`model.train()` uses batch stats; `model.eval()` uses the frozen running stats. Forgetting to switch is a classic bug.

Worked BatchNorm

D

Feature values $x = [1, 2, 3, 4]$ across a batch, with $\gamma = 2, \beta = 1$:

$$\mu = 2.5, \qquad \sigma^2 = 1.25.$$

$$\hat{x} = \frac{x - 2.5}{\sqrt{1.25}} \approx [-1.342, -0.447, 0.447, 1.342].$$

$$y = 2\hat{x} + 1 \approx [-1.683, 0.106, 1.894, 3.683].$$

$\hat{x}$ is mean-0 / var-1; γ, β then place it where the network wants — verified numerically

Why does BatchNorm help? D

The original story was **“reducing internal covariate shift”** — the layer-input distribution moving during training.

Later work found that explanation **does not hold up**: the better-supported reason is that BN makes the optimization **landscape smoother** (well-conditioned), so larger, more stable steps are possible.

Teach it as **smoother optimization + a controlled scale**, not as “fixing covariate shift”.

BatchNorm's limitations

BatchNorm couples every example in the batch:

- **batch-dependent** — an example's output depends on **who else** is in the batch;
- **small-batch noise** — statistics are unreliable for tiny batches;
- **train $\neq$ inference** — needs running stats and a mode switch;
- **awkward for sequences** — variable lengths, per-step statistics.

these are exactly the cases where LayerNorm wins

Normalize **each example over its own features** — no batch involved:

$$\mu_i = \frac{1}{D}\sum_{j=1}^{D} x_{ij}, \quad \sigma_i^2 = \frac{1}{D}\sum_{j=1}^{D} (x_{ij} - \mu_i)^2, \quad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

- **batch-independent** — identical result for batch size 1 or 1000;
- **train $=$ inference** — no running statistics needed.

LayerNorm normalizes across each example

orange row = one example i, normalized using the mean/variance **of its own features**.

reduce **across** each row — the batch axis is never touched

BatchNorm vs LayerNorm

V

Aspect	BatchNorm	LayerNorm
normalize over	batch (per feature)	features (per example)
batch-dependent?	yes	no
train = inference?	no (running stats)	yes
small batch	noisy / unstable	unaffected
sequences	awkward	natural
typical use	CNNs	Transformers, RNNs

RMSNorm

optional

Drop the mean-centring; rescale by the root-mean-square only:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{D}\sum_{j=1}^{D} x_j^2}, \qquad y = \gamma\,\frac{x}{\mathrm{RMS}(x)}.$$

Cheaper than LayerNorm (no mean, no subtraction), and works about as well — common in recent large language models.

Interactive: normalization axes

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/norm-comparison

Toggle Batch / Layer / RMS norm on the same $B \times D$ tensor and watch which axis gets reduced — then compare train-time vs eval-time behaviour for BatchNorm.

Q. Why does LayerNorm give the **same** output at batch size 1 and 128, while BatchNorm does not?

Residual connections: short paths

The degradation problem

V

He et al. (2015): a **56-layer** plain net had **higher training error** than a **20-layer** one.

Not overfitting — the **training** error is worse, not just the test error.

It is an **optimization** failure: the deeper net **cannot even fit** the data the shallow one fits.

deeper should be at least as easy — yet plain stacks get harder to optimize

Learn the residual, not the map D

A deep block should be able to represent the **identity** easily. Reparameterize the target $H(x)$:

$$F(x) := H(x) - x \quad \Rightarrow \quad H(x) = x + F(x).$$

Instead of learning H from scratch, the block learns the **residual** F — the **change** to apply.

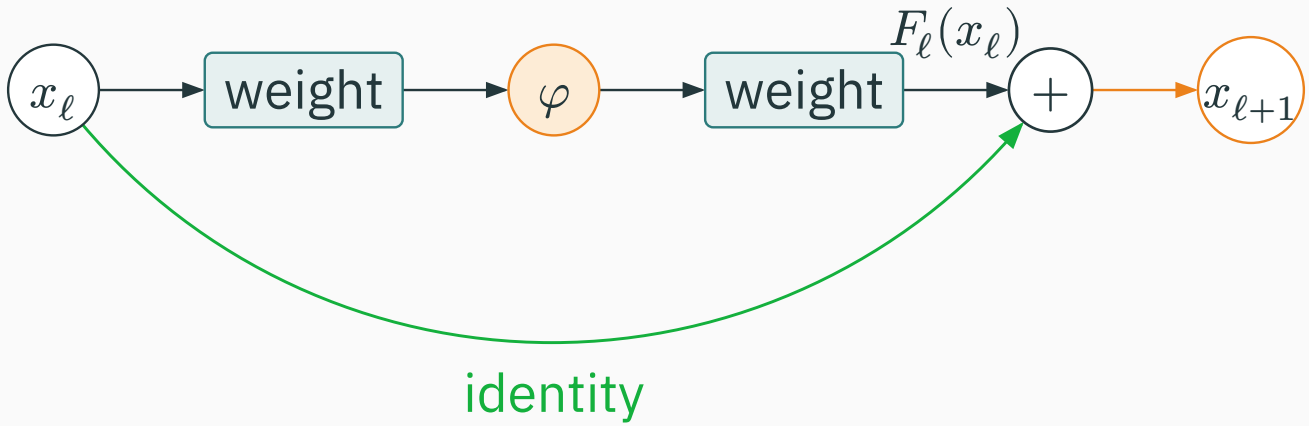
If the best thing to do is "nothing", the block only needs $F \approx 0$ — far easier than learning $H \approx \text{identity}$ from random weights.

The residual block

D

Stack of blocks, each adding its output to its input:

$$x_{\ell+1} = x_\ell + F_\ell(x_\ell).$$



the green identity shortcut carries x_ℓ straight to the addition, skipping the branch

The forward information path

D

Unroll the recurrence from layer ℓ to layer L:

$$x_L = x_\ell + \sum_{i=\ell}^{L-1} F_i(x_i).$$

every layer has a direct, additive contribution to the output

the input reaches the top through the identity path, never multiplied away

The backward gradient path

D

Differentiate $x_{\ell+1} = x_\ell + F_\ell(x_\ell)$:

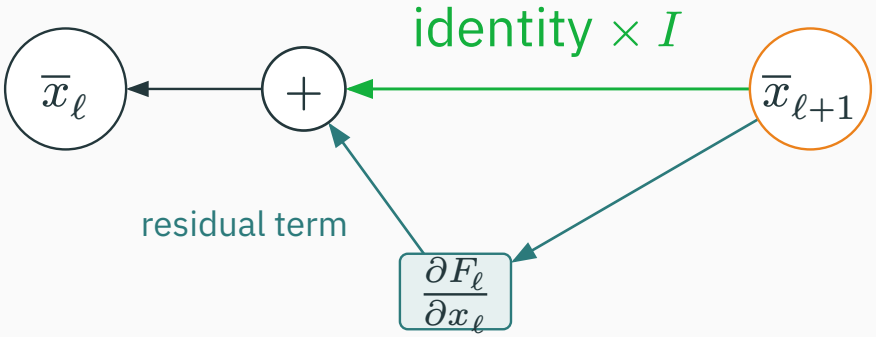
$$\frac{\partial \mathcal{L}}{\partial x_\ell} = \frac{\partial \mathcal{L}}{\partial x_{\ell+1}}\left(I + \frac{\partial F_\ell}{\partial x_\ell}\right) = \underbrace{\frac{\partial \mathcal{L}}{\partial x_{\ell+1}}}_{\text{identity term}} + \underbrace{\frac{\partial \mathcal{L}}{\partial x_{\ell+1}}\frac{\partial F_\ell}{\partial x_\ell}}_{\text{residual term}}.$$

the identity term passes the gradient through with no matrix multiply

even if the branch Jacobian vanishes, the I keeps a clean path back

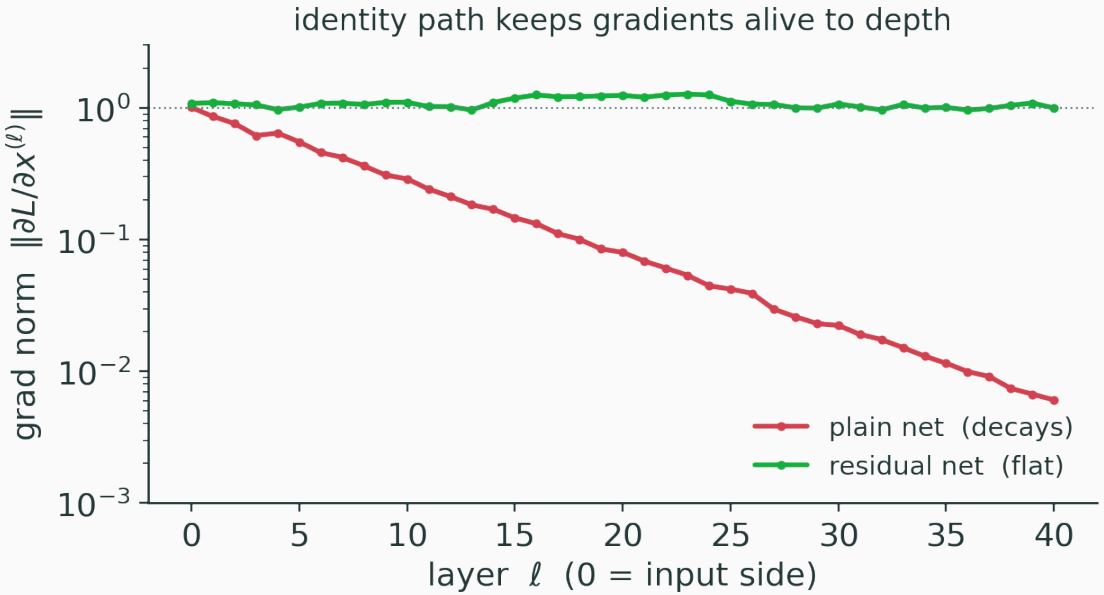
The gradient's two paths

Backward, the upstream gradient $\overline{x}_{\ell+1}$ splits and **adds** at the input:



the green identity path delivers $\overline{x}_{\ell+1}$ untouched — so $\overline{x}_\ell$ stays alive even if the residual term shrinks

Residuals keep gradients alive



plain net: gradient decays with depth · residual net: the identity path holds it flat

Worked scalar residual

D

Take a scalar block $y = x + wx^2$ with the shortcut. Differentiate:

$$\frac{\mathrm{d}y}{\mathrm{d}x} = 1 + 2wx.$$

At $w = 0.1, x = 2$: $1 + 2(0.1)(2) = 1.4$. **Without** the shortcut ($y = wx^2$): $\frac{\mathrm{d}y}{\mathrm{d}x} = 2wx = 0.4$.

the identity keeps a 1 in the gradient — it cannot vanish below that

Dimension-changing shortcuts

D

When F_ℓ changes width or resolution, the identity no longer fits. Project it:

$$x_{\ell+1} = P_\ell\, x_\ell + F_\ell(x_\ell).$$

- P_ℓ is a learned **linear projection** (a 1×1 conv in CNNs, or a stride to downsample);
- used only at the blocks where dimensions change; elsewhere $P_\ell = I$.

Pre- vs post-activation blocks

optional

Where does the activation (and norm) sit relative to the add?

Post-activation (original ResNet):
$x_{\ell+1} = \varphi\big(x_\ell + F_{\ell(x_\ell)}\big)$.

Pre-activation: norm and φ go **inside** F_ℓ, keeping the shortcut a **clean** identity.

pre-activation keeps the skip path unobstructed — better for very deep nets

Residuals are not magic

A shortcut alone does not fix everything:

- the branch still needs sensible **scale and init** (start $F \approx 0$);
- **norm placement** around the block matters (pre- vs post-);
- the **learning rate** still interacts with everything.

Fixup init trains deep ResNets **without** normalization — by carefully scaling the residual branches. Init, norm and residuals are three **coupled** knobs, not independent fixes.

Interactive: residual gradient flow

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/resnet

Toggle the skip connections on and off in a deep net and watch the backward gradient norm go from decaying (plain) to flat (residual), layer by layer.

Q. In $\partial \mathcal{L} / \partial x_\ell$, which term guarantees the gradient never fully vanishes?

Putting it together

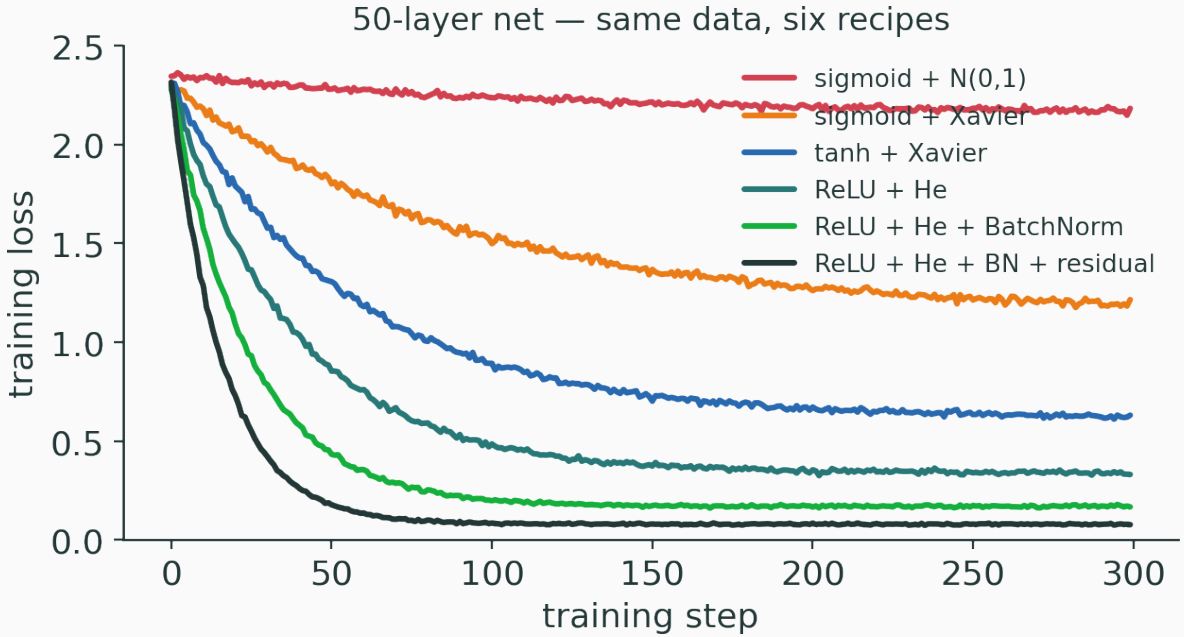
A 50-layer ablation

Same 50-layer net and data, six recipes stacked from naive to modern:

Activation	Init	Extras	Outcome
sigmoid	$\mathcal{N}(0, 1)$	—	dead — no learning
sigmoid	Xavier	—	learns, very slow
tanh	Xavier	—	trains, still slow
ReLU	He	—	trains reliably
ReLU	He	BatchNorm	faster, stable
ReLU	He	BN + residual	fastest, deepest

each intervention removes a different obstacle to optimization

The ablation, as curves



naive sigmoid never moves; each added technique lowers and speeds the loss

What each technique changes

Technique	**What it controls**
initialization	the **starting** distribution of signals
activation	the **local** nonlinearity and its derivative φ'
normalization	the **scale and parameterization** during training
residual	the **path length** for signal and gradient

These knobs are coupled

They are **not** independent tricks:

- ReLU $\Rightarrow$ use **He**, not Xavier (the factor of 2);
- BatchNorm **reduces** but does not **remove** init sensitivity;
- residual branches want **smaller** init (start near 0);
- LayerNorm **placement** changes the residual stream;
- the **learning rate** interacts with all of the above.

Change one and you often have to retune another — treat them as a **system**, not a checklist.

A diagnostic dashboard

I

When a deep net will not train, **log per layer** and look for the outlier:

```
for name, h in activations.items():
    log(name, h.mean(), h.std(), h.min(), h.max())     # signal scale
    log(name, (h == 0).float().mean())                  # % dead (ReLU)
for name, p in model.named_parameters():
    log(name, p.grad.norm() / (p.norm() + 1e-8))        # grad-to-weight ratio
    log(name, (~torch.isfinite(p.grad)).any())          # NaN / Inf guard
```

healthy: std $\approx$ const with depth, grad/weight ratio $\approx 10^{-2}$ to 10^{-3} everywhere, no NaNs

Symptom → issue → fix

Symptom	Likely issue	Fix
activation std $\to 0$ with depth	init too small / saturation	He init, ReLU/GELU
activation std $\to \infty$ / NaN	init too large / no norm	scale down, clip, add norm
high % zero activations	dead ReLU (LR / neg bias)	lower LR, LeakyReLU
tiny grad in early layers	vanishing gradient	residual, norm, ReLU
loss diverges in first steps	LR too high / no warmup	warmup, lower LR, clip

Setting	Sensible default recipe
MLP	ReLU / GELU + He init + (optional LayerNorm)
CNN	He init + ReLU / SiLU + BatchNorm + residual
Transformer	LayerNorm / RMSNorm + residual + GELU / SiLU + warmup

start here, then adjust — these are starting points, not laws

What **not** to overclaim

- BatchNorm does **not** “solve internal covariate shift” — it smooths optimization;
- residuals do **not** prevent **all** vanishing — they add a path, not a guarantee;
- ReLU **can** effectively **die** — watch the zero fraction;
- He does **not** preserve variance **exactly** — it is an approximation that erodes;
- normalization is **not always** required (Fixup, careful init);
- **deeper is not always better** — depth has to be earned.

Retrieval exercise

Q

Say the fix out loud before you read it:

Cue	Recall
tanh / sigmoid	saturation $\Rightarrow$ vanishing gradients
ReLU on the negative side	zero gradient $\Rightarrow$ can die
$\mathrm{Var}(W) = 1/n_{\text{in}}$	Xavier (linear / tanh)
$\mathrm{Var}(W) = 2/n_{\text{in}}$	He (ReLU)
$x + F(x)$	residual block $\Rightarrow$ identity gradient path

Final mental model — four moves

initialization = start in a trainable regime
activations = preserve a nonlinear gradient
normalization = stabilize scale + optimization
residuals = shorten the signal / gradient paths

all four keep signals alive forward and gradients alive backward

Now the network can be **optimized**.

Next: keeping it from fitting the training data too well — **regularization and generalization**.